

Речь к презентации проекта «Навигация и картография робота по линиям»

Перепечаев Виктор, Артур Адамов, Тимофей Вязовик

18 мая 2026 г.

Слайд 1. Титульный слайд

Здравствуйте. Мы представляем проект «Навигация и картография робота по линиям». Его практическая цель — восстановить двумерную траекторию робота по данным камеры ZED и IMU. В реализации это оформлено как Python-библиотека и CLI-пайплайн `awesome-feature-navigation`, который обрабатывает запись офлайн и сохраняет результат в виде CSV и интерактивного HTML-графика.

Слайд 2. Содержание

Сначала я кратко расскажу про команду и постановку задачи. Затем перейду к данным, технологиям и календарному плану. Основная часть будет про реализацию: синхронизацию видео и IMU, детекцию синей ленты, усреднение кругов и объединение траекторий с левой и правой камер. В конце покажу проблемы, которые пришлось решать, сравнение вариантов прогона и дальнейшие направления развития.

Слайд 3. Команда проекта

В команде указаны Виктор Перепечаев, Артур Адамов и Тимофей Вязовик. Тимофей работает над статьей, эта часть отделена от реализации пайплайна. Основная инженерная работа по CV, IMU, алгоритмам и инфраструктуре распределялась между Виктором и Артуром смешанно: задачи перераспределялись по ситуации, без жесткого разделения на полностью независимые зоны. Также важно, что в начале семестра один участник команды отчислился, поэтому нагрузки и приоритеты пришлось перестраивать уже по ходу проекта.

Слайд 4. Постановка проблемы

На входе у нас есть видео с двух камер ZED: `Right_cam.mp4` и `Left_cam.mp4`. Также есть IMU-данные с акселерометром и гироскопом, timestamps кадров и Kalibr-калибровки IMU и camera-IMU. Требуется восстановить траекторию робота во времени в виде координат X , Y и угла φ . То есть задача не просто отрисовать красивую линию, а собрать воспроизводимый путь из нескольких источников измерений.

Слайд 5. Особенности входных данных

У данных есть несколько важных особенностей. Робот едет по синей ленте на полу, и это дает сильную визуальную подсказку. Камера ZED дает два монокулярных потока — левый и правый. Один монокулярный поток сам по себе не дает надежного абсолютного масштаба, но две независимые оценки одной и той же поездки можно потом согласовать. IMU потребительского уровня: гироскоп полезен для yaw, а double integration акселерометра быстро дает дрейф. Отдельная сложность — видео и IMU имеют timestamps в наносекундах, поэтому нужна аккуратная синхронизация по времени.

Слайд 6. Цель проекта

Цель проекта — сделать воспроизводимый офлайн-пайплайн. Это означает, что мы берем уже записанные видео и IMU, прогоняем их одной командой и получаем понятные артефакты: CSV с траекторией, HTML-график и при необходимости отладочные файлы. Мы не заявляем полноценный SLAM с картой и loop closure на уровне factor graph. Задача более прикладная: восстановить траекторию конкретного робота по линии и сделать это достаточно устойчиво.

Слайд 7. Ресурсы и технологии

Основной язык реализации — Python, поддерживаются версии 3.10–3.14. Для зависимостей используется uv. Из computer vision используется OpenCV: HSV-сегментация, distance transform, morphology и optical flow. Для графиков используется Plotly, потому что результат удобно смотреть в браузере. Математическая часть включает NumPy, IMU preintegration, Kalibr-калибровки, SVD, Procrustes alignment и MAD-фильтр. За качество отвечают pytest, ruff, муру и GitHub Actions.

Слайд 8. Календарное планирование

Работа шла по этапам. В феврале мы формулировали задачу, разбирались с данными, Kalibr и форматом ZED. Затем занимались калибровкой камеры и обходом проблемы Kalibr с одной камерой. В марте была IMU-калибровка через AprilGrid, синхронизация времени и базовое интегрирование yaw. В апреле появилась основная ветка `tape_line`: HSV-маска, centerline и интеграция позы. Во второй половине апреля и мае были добавлены offline smoothing, confidence, variable speed correction, loop averaging, ручные границы кругов, а затем fusion левой и правой камер, тесты и документация.

Слайд 9. Общая схема пайплайна

Пайплайн начинается с чтения конфига, CLI-параметров и Kalibr YAML. Затем загружаются IMU CSV и timestamps кадров, после чего время нормализуется. IMU переводится в camera frame, применяется gravity alignment и коррекция yaw bias. Дальше на видео автоматически настраивается HSV-диапазон синей ленты, строятся наблюдения линии, интегрируется поза и получается `raw_traj`. Потом включается сглаживание и loop averaging, чтобы получить устойчивый замкнутый круг. Для двухкамерного сценария такие круги строятся отдельно для LEFT и RIGHT, после чего объединяются в итоговую траекторию.

Слайд 10. Измерение шумов IMU

Перед использованием IMU нужно понимать уровень шума. Для этого была измерена Allan deviation акселерометра по осям X, Y и Z. По наклонам графика оцениваются параметры white noise и random walk. Эти значения попадают в `imu-imu_calibration.yaml` и дальше используются в параметрах IMU preintegration. Даже если в основном рабочем сценарии translation от акселерометра не используется напрямую, корректная калибровка IMU все равно важна для yaw и для общей воспроизводимости пайплайна.

Слайд 11. Режим `tape_line`

Основной режим для нашего видео — `tape_line`. В каждом кадре изображение уменьшается, переводится в HSV, затем по диапазону синего цвета строится бинарная маска. Из маски остается крупнейшая компонента, применяется сглаживание и morphology, а centerline находится через distance transform. Для каждого кадра сохраняются угол линии, положение линии внизу кадра и confidence. После этого наблюдения сглаживаются офлайн по времени, а траектория интегрируется с учетом forward speed, yaw от IMU, yaw correction и lateral correction по положению ленты.

Слайд 12. Усреднение кругов

Так как робот проходит повторяющуюся петлю, один полный путь можно использовать для стабилизации результата. Сначала оценивается период круга или задаются ручные границы через `--lap-bounds`. Затем каждый круг ресэмплируется и выравнивается к шаблону через rigid Procrustes, то есть поворот и перенос без масштабирования. Плохие круги отбрасываются MAD-фильтром и quality gate по RMSE. На выходе получается один representative lap — замкнутый круг, который лучше отражает повторяемую траекторию, чем один шумный проход.

Слайд 13. Слияние LEFT + RIGHT

На этом слайде показана идея двухкамерного усреднения. Левая и правая камеры видят одну и ту же физическую поездку, но из разных оптических центров. Поэтому мы сначала независимо строим траекторию по `Left_cam.mp4` и по `Right_cam.mp4`. Затем команда `afn-fuse` ресэмплирует обе петли к одинаковому числу точек, проверяет прямое и обратное направление LEFT, выравнивает фазу и делает rigid Procrustes alignment LEFT к RIGHT. После этого соответствующие точки усредняются: итоговая координата равна половине RIGHT плюс половине выровненной LEFT. RIGHT здесь задает систему координат вывода, но не считается правильным ответом.

Слайд 14. Проблема 1: третий круг идет против часовой стрелки

Первая практическая проблема связана с направлением движения. В записи первые два круга робот проходит в одном направлении, а третий — в противоположном. Автоматическая нарезка не всегда надежно различает направление обхода. Решение — использовать ручные

--lap-bounds и явно указать направления кругов в конфиге: первые два как `forward`, третий как `reverse`. После этого `reverse`-круг разворачивается по порядку точек и может участвовать в общей `representative lap`.

Слайд 15. Проблема 2: робот стоит первые 11 секунд

Вторая проблема — стационарный старт. Робот в начале стоит примерно 11 секунд, из-за этого автоматический поиск `anchor`-точек может сдвигать границы кругов. Для алгоритма это выглядит как часть траектории, хотя физически движение еще не началось. Решение такое же прагматичное: задавать ручные границы кругов, где первый круг начинается с 11 секунды. Это не подгонка под форму ответа, а фиксация известного временного факта из видео.

Слайд 16. Проблема 3: IMU translation drift

Третья проблема — нельзя напрямую строить `translation` двойным интегрированием `accel`. Ошибка растет примерно как t^2 , и уже за один круг становится сравнимой с размером самой траектории. Поэтому мы используем IMU в основном для `yaw`: это один интеграл, и дрейф там значительно медленнее. Перемещение вперед задается через `forward_speed_mps`, а затем корректируется `variable-speed` механизмом по повторяемости кругов.

Слайд 17. Проблема 4: Kalibr не работает с одной камерой

Еще одна инженерная проблема возникла с Kalibr. В нашем сценарии `intrinsic`-калибровка одной камеры могла падать из-за проверки связности графа в `MulticamGraph.py`. Это известная проблема в Kalibr. Мы использовали `community`-обход: заменить проверку `return self.G.adhesion()` на `return True`. Это позволило завершить калибровочный `pipeline` и получить нужные параметры для дальнейшей работы.

Слайд 18. Проблема 5: IMU-калибровка и инфраструктура

Пятая проблема была не только алгоритмической, но и инфраструктурной. Для IMU-калибровки через `AprilGrid` нужно физически вращать робота по всем осям, чтобы данные были достаточно информативными. Кроме того, часть инструментов ZED, например конвертация SVO2, требует CUDA и NVIDIA GPU, а у команды такого ноутбука не было. Поэтому для конвертации приходилось привлекать внешнюю помощь, в частности ноутбук Роберта Вахненко.

Слайд 19. Сравнение вариантов прогона

Здесь сравниваются варианты: правая камера в автоматическом режиме, правая камера с ручными границами, левая камера с ручными границами и итоговое слияние `LEFT + RIGHT`. В `fused`-варианте длина получается около 11.22 метра, `bbox` — примерно 2.93 на 4.27 метра, `aspect` — 1.46. Дополнительно смотрим качество выравнивания `LEFT` к `RIGHT`: `mean residual` около 0.34 метра, `median` 0.28 метра, `p95` 0.78 метра. Это показывает, что две камеры дают согласованные, но не одинаковые оценки, и их усреднение имеет смысл.

Слайд 20. Итоги

В итоге получился воспроизводимый офлайн-пайплайн `awesome-feature-navigation` для восстановления траектории робота по линии с использованием двух камер ZED и IMU. Мы получили замкнутый круг по записи с несколькими повторениями и добавили усреднение LEFT + RIGHT. Также появилась инженерная обвязка: тесты, типизация, линтер, CI и coverage badge. С точки зрения опыта проект показал, как приходится сочетать sensor fusion, робастную геометрию, калибровки и практические ограничения железа.

Слайд 21. Дальнейшее развитие

Дальше проект можно развивать в трех направлениях. Алгоритмически — добавить pose graph optimization и bundle adjustment. По сенсорам — использовать stereo или depth, чтобы получать метрический масштаб без `forward_speed`, а также добавить wheel odometry как дополнительный prior. По функционалу — читать стрелочные символы на поле, делать сценарии следования за объектом, тестировать работу при разном освещении и интегрироваться с прошлым ROS-стеком.

Слайд 22. Спасибо за внимание

На этом все. Спасибо за внимание, готовы ответить на вопросы.